

JS TOPICS

Here is the revised content without timestamps:

Chapter 1: Data Types

- 1.1 Adding JavaScript
- 1.2 Variables
- 1.3 Data Types
- 1.4 Strings
- 1.5 String Methods
- 1.6 Numbers
- 1.7 Loose Equality vs. Strict Equality
- 1.8 Type Conversion
- 1.9 Arrays
- 1.10 Boolean Values and Comparison Operators

Chapter 2: Control Flow

- 2.1 For Loop
- 2.2 While Loop
- 2.3 Do While Loop
- 2.4 if / else if / else
- 2.5 Nested if Statement
- 2.6 Break and Continue
- 2.7 Logical Operators
- 2.8 Variable and Block Scope
- 2.9 Ternary Operator
- 2.10 Switch Statement

Chapter 3: Functions

- 3.1 Function and Function Expression
- 3.2 Function Parameters and Arguments
- 3.3 Return Statement
- 3.4 Arrow Function
- 3.5 Higher Order Function - Callbacks
- 3.6 Higher Order Function - Returning Functions
- 3.7 IIFE
- 3.8 setTimeout and setInterval
- 3.9 Hoisting

Chapter 4: Objects

- 4.1 Object Introduction
- 4.2 Function vs. Methods
- 4.3 "this" Keyword
- 4.4 for-Each Method
- 4.5 Objects Inside Array
- 4.6 Math Object
- 4.7 Call, Apply, and Bind
- 4.8 Pass by Value and Pass by Reference
- 4.9 for-in Loop

Chapter 5: DOM

- 5.1 DOM Introduction
- 5.2 Query Selector
- 5.3 Other Ways to Access Elements

Chapter 6: DOM - Forms

- 6.1 Submit Event
- 6.2 Regular Expression

Chapter 7: Array Methods

- 7.0 Array Methods
- 7.1 Slice
- 7.2 Splice
- 7.3 At
- 7.4 Map
- 7.5 Filter

Chapter 8: Date

- 8.1 Date and Time

Chapter 9: LocalStorage

- 9.1 Local Storage Introduction

Chapter 10: OOP

- 10.1 Constructor and `new` Operator
- 10.2 Prototypes
- 10.3 Prototypical Inheritance
- 10.5 Setters and Getters

10.6 Static Methods
10.7 Class Inheritance

Chapter 11: Async JavaScript

11.4 Callback Function
11.12 Async/Await without Return Statement
11.13 Error Handling Using Try-Catch

Chapter 12: ES6

12.1 Array De-structuring
12.7 for-of Loop

Chapter 13: Advanced Concepts

14.1 Closures

14.1 BOOTStrap

1. Introduction to JavaScript

What is JavaScript?

JavaScript (JS) is a **high-level**, **dynamic**, and **versatile** programming language. It's primarily used for creating **interactive and dynamic content** on web pages.

- **High-level:** JavaScript is easy to write and understand because it abstracts away complex low-level operations, such as memory management.
- **Dynamic:** JS can change the content of a webpage in real time (without refreshing the page), which allows users to interact with the page, like clicking buttons, filling out forms, or even watching animations.
- **Versatile:** JavaScript is used for a wide variety of purposes such as:
 - Animations (like things moving or changing on the screen).
 - Form validation (checking if the user filled out a form correctly).
 - Interactive maps (like those on Google Maps).

Why JavaScript is important?

JavaScript is one of the core technologies of the web. Along with **HTML** (which defines the page structure) and **CSS** (which defines the page style), JavaScript makes modern websites functional and interactive.

2. History of JavaScript

Who created JavaScript?

JavaScript was created by **Brendan Eich**, a programmer at **Netscape**. He created it in just **10 days** in **1995** to make web pages more interactive and responsive to user actions.

Name Evolution:

1. **Original Name:** Initially, it was called **Mocha**.
2. **Later, it was renamed to LiveScript**, and finally to **JavaScript**. The name change was partly a marketing decision because of the popularity of **Java** at that time, though Java and JavaScript are completely different languages.

Why the name JavaScript?

- The **name "JavaScript"** was chosen to take advantage of Java's popularity during the 90s, but JavaScript is not related to Java in terms of functionality.

Standardization:

- In **1997**, JavaScript was officially standardized as **ECMAScript** by **ECMA International**, a committee dedicated to standardizing programming languages. This ensures that JavaScript is consistent across all web browsers.

Key Milestones:

- **1995**: JavaScript was first introduced in **Netscape Navigator**, one of the earliest popular web browsers.
 - **1997**: **ECMAScript 1** was the first official version of JavaScript.
 - **2015**: **ECMAScript 6 (ES6)** introduced major updates, like **let**, **const**, **arrow functions**, and **classes**.
-

3. JavaScript Keywords

What are Keywords?

Keywords are **reserved words** in JavaScript that have predefined meanings. They cannot be used as variable names or function names.

Types of Keywords:

1. Control Flow Keywords:

- **if, else, switch, case, default, break, continue**
- These keywords control the flow of the program, deciding what should happen next based on conditions or loops.

2. Variable Declarations:

- **var, let, const**
- These are used to declare variables. For example:
 - **let** allows you to change the value of a variable.
 - **const** keeps the value fixed (constant).
 - **var** is the older keyword for declaring variables, but it has certain limitations.

3. Functions and Loops:

- **function, return, for, while, do**
- These keywords are used for creating functions or loops. For example:
 - **function** is used to define a function.
 - **return** is used to exit a function and return a value.

4. Special Purpose Keywords:

- **this**: Refers to the **current object** or **context**.
- **null**: Represents an empty or non-existent value.
- **undefined**: Means a variable has been declared but hasn't been assigned a value yet.
- **new**: Used to create new instances of objects.
- **delete**: Removes a property from an object.
- **typeof**: Tells you the type of a variable.
- **instanceof**: Tests whether an object is an instance of a particular class.

4. Working with JavaScript Files

JavaScript File Extension:

JavaScript code is typically saved in files with the `.js` extension. For example: **script.js**.

Linking JavaScript with HTML:

To use JavaScript in a webpage, you can link an external JavaScript file using the `<script>` tag in the HTML file:

```
<script src="script.js"></script>
```

This line tells the browser to fetch the `script.js` file and execute it.

Embedding JavaScript in HTML:

You can also **embed** JavaScript directly inside an HTML document using the `<script>` tag:

```
<script>  
    alert("Hello, World!");  
</script>
```

```
</script>
```

This will display a popup message when the page is loaded.

5. Literals in JavaScript

What are Literals?

Literals are **fixed values** used directly in the code. They can be of different types:

Types of Literals:

Numeric Literals: Numbers like 10, 3.14, etc.

```
let number = 10;
```

String Literals: Text wrapped in either **single quotes** or **double quotes**:

```
let greeting = "Hello, World!";
```

Boolean Literals: Represent true or false:

```
let isActive = true;
```

```
let isAvailable = false;
```

Object Literals: Represent objects using {}:

```
const person = { name: "John", age: 30 };
```

Array Literals: Represent arrays using []:

```
const numbers = [1, 2, 3, 4];
```

6. Constants in JavaScript

What is a Constant?

A **constant** is a variable whose value **cannot be changed** once it's set. You declare constants using the `const` keyword.

Declaring a Constant:

```
const PI = 3.14;
```

- The value of `PI` cannot be changed. For example:

```
PI = 3.1416; // This will cause an error because PI is a constant.
```

Features of Constants:

- A constant must be initialized at the time of declaration.
- Constants are useful for **values that should not change**, like **PI**, **configuration settings**, or **fixed data**.

Explanation ➔ [Click me](#)

Yeh raha **Static** aur **Dynamic** websites ka difference ek **table** mein! 🎉✨

Feature	Static Website 🌐	Dynamic Website 💻
Content	Fixed, same every time	Changes based on user interaction
Technology Used	HTML, CSS	HTML, CSS, JavaScript, Server-side technologies (PHP, Node.js, etc.)
Interaction	No interaction with the user	User interactions (e.g., button clicks, form submissions)
Content Update	Manually update in the code	Automatically updates (via JS, server requests, etc.)
Speed	Faster (less complexity)	Slightly slower due to server requests and data processing
Customization	Limited, no personalization	High, can personalize based on user data or actions
Example	Basic websites, blogs, portfolios	Social media sites, e-commerce, interactive dashboards

Chapter 1: Data Types

1.1 Adding JavaScript

Theory: JavaScript can be added to HTML documents in three primary ways:

1. **Inline JavaScript**: Directly within HTML elements using the *((onclick))*, *(onload)*, and other event attributes.
2. **Internal JavaScript**: Within the `<script>` tag inside the HTML document.

3. **External JavaScript**: In separate .js files linked to the HTML document using the `<script src="script.js"></script>` tag.

Examples:

- **Inline JavaScript:**

```
<button onclick="alert('Hello, World!')">Click  
Me</button>
```

```
• Internal JavaScript:

Html

<!DOCTYPE html>
<html>
<head>
  <title>Internal JavaScript Example</title>
  <script>
    function showAlert() {
      alert('Hello, World!');
    }
  </script>
</head>
<body>
  <button onclick="showAlert()">Click Me</button>
</body>
</html>
```



1.2 Variables

Notes:

- Variables store data and can be declared using `var`, `let`, or `const`.
- `let` and `const` are block-scoped (introduced in ES6), while `var` is function-scoped.

Code and Explanation:

1. Declaring Variables:

```
let name = "Alice"; // Modern way
const PI = 3.14;     // Constant value
var age = 25;        // Older way
console.log(name, age, PI);
```

2. Differences:

- `let`: Can be reassigned but not redeclared within the same block.
- `const`: Cannot be reassigned.
- `var`: Can be reassigned and redeclared, but it's less preferred due to potential bugs.

1.3 Data Types

Notes:

- JavaScript has **primitive types** and **non-primitive types**:
 - **Primitive**: string, number, boolean, null, undefined, symbol, bigint.
 - **Non-primitive**: object (includes arrays, functions, etc.).

Code and Explanation:

```
let aString = "Hello"; // String
let aNumber = 42;      // Number
let aBoolean = true;   // Boolean
let anUndefined;       // Undefined
let aNull = null;      // Null
let anObject = { name: "Alice" }; // Object

console.log(typeof aString); // "string"
console.log(typeof aNumber); // "number"
console.log(typeof anObject); // "object"
console.log(typeof anUndefined); // "undefined"
```

Chapter 3: Functions

Functions are a fundamental building block in JavaScript. They are reusable blocks of code designed to perform specific tasks.

3.1 Function and Function Expression

Theory:

- **Function Declaration**: A standard way to define a function using the `function` keyword.
- **Function Expression**: Assigns a function (can be anonymous) to a variable.

Code:

```
// Function Declaration
function greet() {
  console.log("Hello, World!");
}
greet(); // Output: Hello, World!

// Function Expression
const sayHello = function () {
  console.log("Hello from Function Expression!");
};
```

```
sayHello(); // Output: Hello from Function Expression
```

- **Key Difference:** Function declarations are **hoisted**, meaning they can be called before they are defined. Function expressions are not.

3.2 Function Parameters and Arguments

Theory:

- **Parameters:** Variables listed in the function definition.
- **Arguments:** Values passed to the function when it's invoked.
- Functions can have **default parameters**, which assign a default value if no argument is provided.

Code:

```
// Function with parameters
function multiply(a, b = 1) { // b has a default value of 1
  return a * b;
}

console.log(multiply(5, 10)); // Output: 50
console.log(multiply(5));    // Output: 5 (b defaults to 1)
```

3.3 Return Statement

Theory:

- The **return** statement sends a value back to the function caller and terminates the function execution.

Code:

```
function add(a, b) {
  return a + b; // Returns the sum of a and b
}

const result = add(3, 7);
console.log(result); // Output: 10

function noReturn() {
  console.log("No return here!");
}
```

```
}  
console.log(noReturn()); // Output: "No return here!" followed by undefined
```

3.4 Arrow Function

Theory:

- Arrow functions, introduced in ES6, provide a shorter syntax.
- Arrow functions do not have their own `this` context, making them unsuitable for methods.

Code:

```
// Regular Function  
const square = function (x) {  
    return x * x;  
};  
  
// Arrow Function  
const squareArrow = (x) => x * x;  
  
console.log(square(4)); // Output: 16  
console.log(squareArrow(4)); // Output: 16  
  
// Without parameters  
const greet = () => console.log("Hello!");  
greet(); // Output: Hello!
```

3.5 Higher-Order Function - Callbacks

Theory:

- A Higher-Order Function takes another function as an argument.
- A Callback Function is a function passed into another function as an argument.

Code:

```
function greet(name) {  
    console.log(`Hello, ${name}!`);  
}  
  
function executeCallback(callback, arg) {  
    callback(arg); // Execute the passed function  
}  
  
executeCallback(greet, "Alice"); // Output: Hello, Alice!
```

3.7 IIFE (Immediately Invoked Function Expression)

Theory:

- IIFE executes immediately after it's defined.
- Useful for creating a private scope to avoid polluting the global namespace.

Code:

```
(function () {  
  console.log("This is an IIFE!");  
})();           // Output: This is an IIFE!  
  
                // With parameters  
(function (name) {  
  console.log(`Hello, ${name}!`);  
})("Alice");    // Output: Hello, Alice!
```

3.8 setTimeout and setInterval

setTimeout Theory:

- Executes a function after a specified delay (in milliseconds).

setTimeout Code:

```
console.log("Start");  
  
setTimeout(() => {  
  console.log("Executed after 2 seconds");  
}, 2000);  
  
console.log("End");  
  
// Output:  
// Start  
// End  
// Executed after 2 seconds
```

setInterval Theory:

- Repeatedly executes a function at specified intervals.

setInterval Code:

```
let counter = 0;

const intervalId = setInterval(() => {
  console.log(`Counter: ${++counter}`);
  if (counter === 5) clearInterval(intervalId); // Stops after 5 iterations
}, 1000);

// Output:
// Counter: 1
// Counter: 2
// Counter: 3
// Counter: 4
// Counter: 5
```

3.9 Hoisting

Theory:

- **Hoisting** moves function declarations and variable declarations to the top of their scope during the compilation phase.
- Function expressions and variables using `let` or `const` are not hoisted.

Code:

```
// Function Declaration Hoisting
console.log(hello()); // Output: "Hello!"

function hello() {
  return "Hello!";
}

// Function Expression (Not Hoisted)
console.log(sayHi); // Output: undefined

var sayHi = function () {
  return "Hi!";
};

// let and const Hoisting
let myVar = 10;

console.log(myVar); // ReferenceError: Cannot access 'myVar' before
initialization
```

Summary:

- Functions are reusable blocks of code.
- **Function Declarations** are hoisted, while **Function Expressions** are not.

- Arrow functions offer concise syntax and do not have their own `this`.
- Higher-order functions allow for dynamic programming through callbacks and function returns.
- Tools like **setTimeout** and **setInterval** enable asynchronous code execution.
- Hoisting affects how and when variables and functions are initialized.

Chapter 4: Objects

In JavaScript, **objects** are a collection of key-value pairs. They can store multiple values of different types and are essential for working with structured data. Objects are used extensively in JavaScript to represent real-world entities and more.

4.1 Object Introduction

Theory:

- Objects are a collection of properties (key-value pairs).
- Keys are strings (or symbols), and values can be any valid data type: primitive values, functions, or even other objects.

Code:

```
// Creating an object

const person = {
  name: "John",
  age: 30,
  greet: function () {
    console.log("Hello, " + this.name);
  }
};

console.log(person.name); // Output: John
console.log(person.age);  // Output: 30
person.greet();           // Output: Hello, John
```

4.2 Function vs. Methods

Theory:

- A **function** is a block of code designed to perform a task.
- A **method** is a function that is a property of an object.
- Functions can exist independently, while methods are associated with specific objects.

Code:


```
// Function

function greet() {

    console.log("Hello!");
}

greet(); // Output: Hello!

// Object with Method

const person = {
    name: "Alice",
    greet: function () {
        console.log("Hello, " + this.name);
    }
};
person.greet(); // Output: Hello, Alice
```

4.3 "this" Keyword

Theory:

- The `this` keyword refers to the context of the function where it is called.
- In a regular function, `this` refers to the object from which the function was called.
- In an arrow function, `this` is lexically inherited from the surrounding context.

Code:

```
// Regular function
const person = {
    name: "Alice",
    greet: function () {
        console.log("Hello, " + this.name); // 'this' refers to the person object
    }
};
person.greet(); // Output: Hello, Alice

// Arrow function
const personArrow = {
    name: "Bob",
    greet: () => {
        console.log("Hello, " + this.name); // 'this' refers to the outer scope,
        not the personArrow object
    }
};
personArrow.greet();

// Output: Hello, undefined (as 'this' refers to the global context in non-
methods)
```

4.4 forEach Method

Theory:

- `forEach()` is an array method used to execute a provided function once for each element in the array.
- It does not return a value (undefined is returned).

Code:

```
const numbers = [1, 2, 3, 4, 5];
```

```
numbers.forEach(function (num) {  
  console.log(num * 2);  
});
```

```
// Output:
```

```
// 2
```

```
// 4
```

```
// 6
```

```
// 8
```

```
// 10
```

You can also use an arrow function:

```
numbers.forEach(num => console.log(num * 2));
```

```
// Output: 2, 4, 6, 8, 10
```

4.5 Objects Inside Array

Theory:

- You can store objects within an array.
- Arrays of objects are useful for organizing complex data structures like lists of people, products, etc.

Code:

```
const students = [  
  { name: "John", age: 22 },  
  { name: "Jane", age: 23 },  
  { name: "Mike", age: 21 }  
];
```

```
students.forEach(student => {  
  
  console.log(`${student.name} is ${student.age} years old`);  
});
```

```
// Output:
```

```
// John is 22 years old
```

```
// Jane is 23 years old
// Mike is 21 years old
```

4.6 Math Object

Theory:

- The **Math** object provides properties and methods for mathematical constants and functions.
- It is a built-in object and does not need to be instantiated.

Code:

```
console.log(Math.PI);           // Output: 3.141592653589793
console.log(Math.random());     // Output: A random number between 0 and 1
console.log(Math.pow(2, 3));    // Output: 8 (2 raised to the power of 3)

console.log(Math.sqrt(25));     // Output: 5 (square root of 25)
console.log(Math.round(4.5));   // Output: 5 (rounds to nearest integer)
```

4.7 Call, Apply, and Bind

Theory:

- These methods allow you to invoke functions with a specific `this` context.
 - **call()**: Invokes the function with a specified `this` value and arguments.
 - **apply()**: Similar to `call()`, but arguments are passed as an array.
 - **bind()**: Returns a new function with a specific `this` value, but does not invoke it immediately.

Code:

```
// Using call()
const person = { name: "Alice" };
function greet(greeting) {
  console.log(greeting + ", " + this.name);
}
greet.call(person, "Hello"); // Output: Hello, Alice

// Using apply()
greet.apply(person, ["Hi"]); // Output: Hi, Alice

// Using bind()
const greetPerson = greet.bind(person, "Hey");
greetPerson(); // Output: Hey, Alice
```

4.8 Pass by Value and Pass by Reference

Theory:

- **Pass by Value:** When primitive values (e.g., numbers, strings, booleans) are passed to a function, they are copied, and changes inside the function do not affect the original value.
- **Pass by Reference:** When objects are passed to a function, the reference (memory address) of the object is passed, meaning changes affect the original object.

Code:

```
// Pass by Value
let a = 10;
function changeValue(x) {
  x = 20;
}
changeValue(a);
console.log(a); // Output: 10 (the original value is unchanged)

// Pass by Reference
let person = { name: "John", age: 25 };
function changePerson(obj) {
  obj.age = 26;
}
changePerson(person);
console.log(person.age); // Output: 26 (the original object is modified)
```

4.9 for-in Loop

Theory:

- The `for-in` loop is used to iterate over the properties of an object.
- It loops through all enumerable properties of the object, including those that are inherited through the prototype chain.

Code:

```
const person = {
  name: "Alice",
  age: 25,
  greet: function () {
    console.log("Hello!");
  }
};

for (let key in person) {
  console.log(key + ": " + person[key]);
}
// Output:
// name: Alice
// age: 25
// greet: function() { console.log("Hello!"); }
```

```
// To avoid inherited properties, you can use hasOwnProperty()
for (let key in person) {

    if (person.hasOwnProperty(key)) {

        console.log(key + ": " + person[key]);
    }
}
```

Chapter 5: DOM (Document Object Model)

Chapter 5: DOM

5.1 DOM Introduction

5.2 Query Selector

5.3 Other Ways to Access Elements

The DOM (Document Object Model) is a programming interface for web documents. It represents the page so that programs can manipulate the structure, style, and content of web pages. The DOM represents a page as a tree structure where each node is an object representing a part of the page.

5.1 DOM Introduction

Theory:

- The **DOM** allows JavaScript to access and update the content, structure, and style of HTML documents.
- JavaScript interacts with the DOM to dynamically update web pages without requiring a page reload.
- The DOM represents the HTML structure as a hierarchy of nodes (elements, attributes, text, etc.).

Code Example:

```
<!DOCTYPE html>
<html lang="en">
```

```

<head>
  <meta charset="UTF-8">
  <title>DOM Example</title>
</head>
<body>
  <div id="myDiv">Hello, DOM!</div>

  <script>

    // Accessing the DOM element

    const div = document.getElementById("myDiv");
    console.log(div.textContent); // Output: Hello, DOM!

    // Modifying the DOM element"

    div.textContent = "Hello, JavaScript!";

  </script>
</body>
</html>

```

5.2 Query Selector

Theory:

- `querySelector()` is a method used to select an element from the DOM using CSS selectors.
- It returns the first matching element, or `null` if no matching element is found.

Code Example:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Query Selector Example</title>
</head>
<body>

  <p class="message">This is a paragraph.</p>
  <p id="intro">Another paragraph.</p>

  <script>

    // Using querySelector to select by class

    const message = document.querySelector(".message");
    console.log(message.textContent); // Output: This is a paragraph.

    // Using querySelector to select by ID

    const intro = document.querySelector("#intro");
    console.log(intro.textContent); // Output: Another paragraph.

  </script>
</body>
</html>

```

Code Example:

```
html
Copy code
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>DOM Example</title>
</head>
<body>
  <div id="myDiv">Hello, DOM!</div>

  <script>
    // Accessing the DOM element
    const div = document.getElementById("myDiv");
    console.log(div.textContent); // Output: Hello, DOM!

    // Modifying the DOM element
    div.textContent = "Hello, JavaScript!";
  </script>
</body>
</html>
```

5.2 Query Selector

Theory:

- `querySelector()` is a method used to select an element from the DOM using CSS selectors.
- It returns the first matching element, or `null` if no matching element is found.

Code Example:

```
html
Copy code
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Query Selector Example</title>
</head>
<body>
  <p class="message">This is a paragraph.</p>
  <p id="intro">Another paragraph.</p>

  <script>
    // Using querySelector to select by class
    const message = document.querySelector(".message");
    console.log(message.textContent); // Output: This is a paragraph.

    // Using querySelector to select by ID
    const intro = document.querySelector("#intro");
    console.log(intro.textContent); // Output: Another paragraph.
  </script>
</body>
</html>
```

5.3 Other Ways to Access Elements

Theory:

Besides `querySelector()`, there are other methods to access DOM elements:

- `getElementById()`: Selects an element by its ID.
- `getElementsByClassName()`: Selects elements by their class name.
- `getElementsByTagName()`: Selects elements by their tag name.

Code Example:

```
html
Copy code
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Access Methods</title>
</head>
<body>
  <div id="container">
    <p class="text">First paragraph.</p>
    <p class="text">Second paragraph.</p>
  </div>

  <script>
    // Using getElementById
    const container = document.getElementById("container");
    console.log(container);

    // Using getElementsByClassName (returns a live HTMLCollection)
    const paragraphs = document.getElementsByClassName("text");
    console.log(paragraphs[0].textContent); // Output: First paragraph.
  </script>
</body>
</html>
```

Chapter 4: Objects

4.1 Object Introduction

4.2 Function vs. Methods

4.3 "this" Keyword

4.4 for-Each Method

4.5 Objects Inside Array

4.6 Math Object

4.7 Call, Apply, and Bind

4.8 Pass by Value and Pass by Reference

4.9 for-in Loop

Chapter 4: Objects

4.1 Object Introduction

Definition: An object is a collection of properties, where each property is defined as a key-value pair. Objects can store data and functions, making them essential for organizing complex data structures in JavaScript.

Code Example

```
1 let person = {  
2   name: "John",  
3   age: 30,  
4   greet: function() {  
5     console.log("Hello, " + this.name);  
6   }  
7 };  
8  
9 person.greet(); // Output: Hello, John
```

4.2 Function vs. Methods

Definition: A function is a standalone block of code that can be executed independently, while a method is a function that is associated with an object. Methods can manipulate the object's properties and provide behavior specific to that object.

```

1 function sayHello() {
2     console.log("Hello!");
3 }
4
5 let obj = {
6     sayHello: function() {
7         console.log("Hello from method!");
8     }
9 };
10
11 sayHello(); // Output: Hello!
12 obj.sayHello(); // Output: Hello from method!

```

4.3 "this" Keyword

Definition: The `this` keyword refers to the current object in which the code is executing. Its value can change based on the context in which a function is called, making it crucial for object-oriented programming.

```

1 let car = {
2     brand: "Toyota",
3     getBrand: function() {
4         return this.brand;
5     }
6 };
7
8 console.log(car.getBrand()); // Output: Toyota

```

4.4 for-Each Method

Definition: The `forEach` method is an array method that executes a provided function once for each array element. It simplifies the process of iterating over arrays and is often preferred for its readability.

```

1 let numbers = [1, 2, 3, 4, 5];
2 numbers.forEach(function(num) {
3     console.log(num * 2);
4 });
5 // Output: 2, 4, 6, 8, 10

```

4.5 Objects Inside Array

Definition: Arrays can contain objects as their elements, allowing for the creation of complex data structures. This is useful for representing collections of related data, such as a list of students with their attributes.

```
1 let students = [  
2   { name: "Alice", age: 20 },  
3   { name: "Bob", age: 22 }  
4 ];  
5  
6 students.forEach(function(student) {  
7   console.log(student.name + " is " + student.age + " years old.");  
8 });  
9 // Output: Alice is 20 years old.  
10 //         Bob is 22 years old.
```

4.6 Math Object

Definition: The Math object provides a set of mathematical functions and constants that are built into JavaScript. It allows developers to perform complex calculations without needing to implement these functions manually.

```
1 let radius = 5;  
2 let area = Math.PI * Math.pow(radius, 2);  
3 console.log("Area of the circle: " + area); // Output: Area of the ci
```

4.7 Call, Apply, and Bind

Definition: These methods allow you to control the value of `this` in functions. `call` and `apply` invoke a function with a specified `this` value, while `bind` creates a new function with a fixed `this` value, which can be called later.

```

1 function greet(greeting) {
2     console.log(greeting + ", " + this.name);
3 }
4
5 let person = { name: "John" };
6
7 greet.call(person, "Hello"); // Output: Hello, John
8 greet.apply(person, ["Hi"]); // Output: Hi, John
9
10 let greetJohn = greet.bind(person);
11 greetJohn("Hey"); // Output: Hey, John

```

4.8 Pass by Value and Pass by Reference

Definition: In JavaScript, primitive data types (like numbers and strings) are passed by value, meaning a copy is made. In contrast, objects and arrays are passed by reference, allowing functions to modify the original object.

```

1 let num = 10;
2 function changeValue(n) {
3     n = 20; // This does not change the original num
4 }
5 changeValue(num);
6 console.log(num); // Output: 10
7
8 let obj = { value: 10 };
9 function changeObject(o) {
10     o.value = 20; // This changes the original object
11 }
12 changeObject(obj);
13 console.log(obj.value); // Output: 20

```

4.9 The `for-in` loop :

iterates over all enumerable properties of an object, including those inherited from its prototype chain.

It is important to use the `hasOwnProperty` method to filter out properties that are inherited from the prototype chain if you only want to work with the object's own properties.

```
1 let car = {  
2   brand: "Toyota",  
3   model: "Camry",  
4   year: 2020  
5 };  
6  
7 // Using for-in loop to iterate over the properties of the car object  
8 for (let key in car) {  
9   console.log(key + ": " + car[key]);  
10 }
```

```
1 brand: Toyota  
2 model: Camry  
3 year: 2020
```

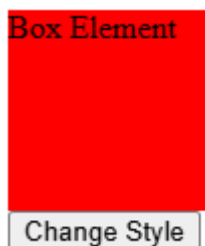
TASK →

Style color :

```
index.html      styles.css      script.js      +      🖼️
1  <!DOCTYPE html>
2  <html lang="en">
3  <body>
4  <div id="box" style="width: 100px; height: 100px; background-color: blue;">
5      Box Element
6  </div>
7  <button id="changeButton">Change Style</button>
8
9  <script>
10     const box = document.getElementById('box');
11
12     // Change the background color of the box when the button is clicked
13     document.getElementById('changeButton').addEventListener('click', function() {
14         box.style.backgroundColor = 'red'; // Changes background color to red
15     });
16 </script>
17 </body>
18 </html>
19
```



After click :->



In addition to the common methods like `getElementById()`, there are other ways to access elements in the DOM. These methods are useful depending on what you're trying to achieve.

1. *getElementsByClassName(class)*

- **Explanation:**

This method returns a **live collection** of elements with the given class name. The collection is "live," meaning it updates automatically when the DOM changes.

```
<!DOCTYPE html>
<html lang="en">
<body>
  <div class="box">Box 1</div>
  <div class="box">Box 2</div>
  <div class="box">Box 3</div>

  <script>
    const boxes = document.getElementsByClassName('box');
    console.log(boxes); // Will return a live HTMLCollection

    // You can loop through the collection and log each box
    for (let i = 0; i < boxes.length; i++) {
      console.log(boxes[i].textContent);
    }
  </script>
</body>
</html>
```

Box 1
Box 2
Box 3

```
Box 1 43376fb8z_4338ftk5x/:14
Box 2 43376fb8z_4338ftk5x/:14
Box 3 43376fb8z_4338ftk5x/:14
```

2. `getElementsByTagName(tag)`

- **Explanation:**

This method returns a **live collection** of all elements with the specified tag name (e.g., `div`, `p`, etc.).

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <body>
4   <p>This is paragraph 1.</p>
5   <p>This is paragraph 2.</p>
6   <p>This is paragraph 3.</p>
7
8   <script>
9     const paragraphs = document.getElementsByTagName('p');
10    console.log(paragraphs); // Returns a live HTMLCollection
11
12    // Loop through paragraphs
13    for (let i = 0; i < paragraphs.length; i++) {
14      console.log(paragraphs[i].textContent);
15    }
16  </script>
17 </body>
18 </html>
19
```

This is paragraph 1.
This is paragraph 2.
This is paragraph 3.

This [43376ff8e_4338fwbp/14](#)
is paragraph 1.

This [43376ff8e_4338fwbp/14](#)
is paragraph 2.

This [43376ff8e_4338fwbp/14](#)
is paragraph 3.

Chapter 6: DOM - Forms

6.1 Submit Event

6.2 Regular Expression

6.3 Basic Form Validation

6.4 Keyboard Event

6.1 Submit Event

Theory: The `submit` event is triggered when a form is submitted. This event can be used to execute JavaScript when the form is submitted, such as validating form data before sending it to the server.

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Submit Event Example</title>
7 </head>
8 <body>
9   <form id="myForm">
10     <label for="name">Name:</label>
11     <input type="text" id="name" name="name" required>
12     <button type="submit">Submit</button>
13   </form>
14
15   <script>
16     document.getElementById('myForm').addEventListener('submit', function(event) {
17       alert('Form submitted successfully!');
18       // Prevent the default form submission
19       event.preventDefault();
20     });
21   </script>
22 </body>
23 </html>
24
```

Name:

An embedded page at app.onecompiler.com says

Form submitted successfully!

OK

Example →

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <title>Submit Event Example</title>
5 </head>
6 <body>
7   <form id="myForm">
8     <label for="name">Name:</label>
9     <input type="text" id="name" name="name" required>
10    <br><br>
11    <label for="email">Email:</label>
12    <input type="email" id="email" name="email" required>
13    <br><br>
14    <button type="submit">Submit</button>
15  </form>
16  <script>
17    document.getElementById('myForm').addEventListener('submit', function(event) {
18      // Prevent the default form submission
19      event.preventDefault();
20
21      // Get the form data
22      const name = document.getElementById('name').value;
23      const email = document.getElementById('email').value;
24
25      // Print form data to the console
26      console.log('Name:', name);
27      console.log('Email:', email);
28      // Show an alert with the form data
29      alert('Form submitted successfully!\nName: ' + name + '\nEmail: ' + email);
30    });
31  </script>
32 </body>
33 </html>
34
```

Name:

Email:

An embedded page at app.onecompiler.com says

Form submitted successfully!

Name: some

Email: kerigi2149@kelenson.com

OK

Name: some

Email: kerigi2149@kelenson.com

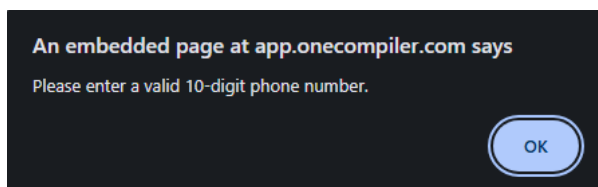
6.2 Regular Expression

Theory: Regular expressions (regex) are patterns used to match character combinations in strings. They are often used for form validation to ensure that user input matches a specific pattern, such as an email address or phone number.

Phone Number →

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Phone Number Validation Example</title>
7 </head>
8 <body>
9   <form id="phoneForm">
10     <label for="phone">Phone Number:</label>
11     <input type="text" id="phone" name="phone" required>
12     <button type="submit">Submit</button>
13   </form>
14
15   <script>
16     document.getElementById('phoneForm').addEventListener('submit', function(event) {
17       const phone = document.getElementById('phone').value;
18       // Regular expression for phone number validation
19       const phonePattern = /^d{10}$/;
20
21       if (!phonePattern.test(phone)) {
22         alert('Please enter a valid 10-digit phone number.');
```

Incorrect



Correct

Phone Number:

An embedded page at app.onecompiler.com says

Phone number is valid!

OK

6.3 Basic Form Validation

Theory: Basic form validation is a way to ensure that user input meets certain criteria before the form is submitted. This includes checking if required fields are filled, if input values match specific patterns, and if values fall within a particular range. This helps in preventing incorrect or incomplete data submission.

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Basic Form Validation Example</title>
7 </head>
8 <body>
9   <form id="validationForm">
10     <label for="username">Username:</label>
11     <input type="text" id="username" name="username" required>
12     <br><br>
13     <label for="password">Password:</label>
14     <input type="password" id="password" name="password" required>
15     <br><br>
16     <label for="email">Email:</label>
17     <input type="email" id="email" name="email" required>
18     <br><br>
19     <button type="submit">Submit</button>
20   </form>
21   <script>
22     document.getElementById('validationForm').addEventListener('submit', function(event) {
23       // Prevent the default form submission
24       event.preventDefault();
25       // Get the form data
26       const username = document.getElementById('username').value;
27       const password = document.getElementById('password').value;
28       const email = document.getElementById('email').value;
```

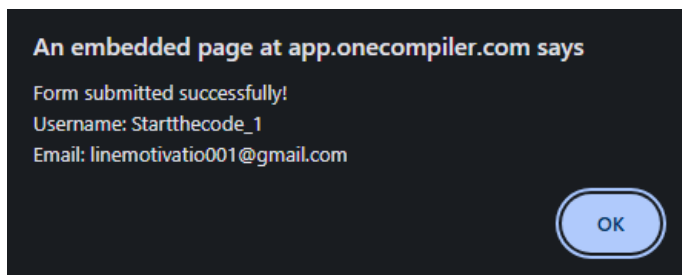
```

21 <script>
22 document.getElementById('validationForm').addEventListener('submit', function(event) {
23     event.preventDefault();
24     // Get the form data
25     const username = document.getElementById('username').value;
26     const password = document.getElementById('password').value;
27     const email = document.getElementById('email').value;
28
29     // Basic validation checks
30     if (username === '' || password === '' || email === '') {
31         alert('All fields are required!');
32         return;
33     }
34     // Password length check
35     if (password.length < 6) {
36         alert('Password must be at least 6 characters long.');
```

Username:

Password:

Email:



6.4 Keyboard Event

Theory: Keyboard events are events triggered by keyboard actions, such as pressing, releasing, or holding down a key. Common keyboard events include `keydown`, `keyup`, and `keypress`. These events are useful for capturing user input, creating keyboard shortcuts, and enhancing user interactions on a webpage.

Common Keyboard Events:

1. **keydown**: Triggered when a key is pressed down.
2. **keyup**: Triggered when a key is released.
3. **keypress**: Triggered when a key is pressed down and that key produces a character value (deprecated, `keydown` is preferred).

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Keyboard Event Example</title>
7   <style>
8     #output {
9       margin-top: 20px;
10      padding: 10px;
11      border: 1px solid #ccc;
12      background-color: #f9f9f9;
13    }
14  </style>
15 </head>
16 <body>
17   <h1>Press Any Key</h1>
18   <div id="output">Key Pressed: </div>
19
20   <script>
21     document.addEventListener('keydown', function(event) {
22       const output = document.getElementById('output');
23       output.textContent = 'Key Pressed: ' + event.key;
24     });
25   </script>
26 </body>
27 </html>
28
```

Detect All Keyword keys

Press Any Key

Key Pressed: Shift

Press Any Key

Key Pressed: Shift

Chapter 7: Array Methods

7.0 Array Methods

- 7.1 Slice
- 7.2 Splice
- 7.3 At
- 7.4 Map
- 7.5 Filter

7.0 Array Methods

Array methods in JavaScript are functions that can be called on arrays to perform various operations. They help in manipulating and processing array data efficiently.

7.1 Slice

Theory: The `slice()` method returns a shallow copy of a portion of an array into a new array object. It takes two arguments: start and end (optional). The end index is not included in the new array.

Example:

```
const fruits = ['apple', 'banana', 'cherry', 'date'];
const slicedFruits = fruits.slice(1, 3);

console.log(slicedFruits);

// Output: ['banana', 'cherry']
```

7.2 Splice

Theory: The `splice()` method changes the contents of an array by removing or replacing existing elements and/or adding new elements in place. It takes at least two arguments: start and deleteCount.

Example:

Javascript →

```
const fruits = ['apple', 'banana', 'cherry', 'date'];
const removedFruits = fruits.splice(1, 2, 'blueberry', 'citrus');

console.log(fruits);

// Output: ['apple', 'blueberry', 'citrus', 'date']

console.log(removedFruits);

// Output: ['banana', 'cherry']
```

7.3 At

Theory: The `at()` method takes an integer value and returns the item at that index. It supports negative integers to count back from the last item in the array.

Example:

javascript

```
const fruits = ['apple', 'banana', 'cherry', 'date'];
console.log(fruits.at(1));           // Output: 'banana'
console.log(fruits.at(-1));          // Output: 'date'
```

7.4 Map

Theory: The `map()` method creates a new array populated with the results of calling a provided function on every element in the calling array.

Example:

javascript

```
const numbers = [1, 2, 3, 4];
const doubled = numbers.map(num => num * 2);
console.log(doubled);           // Output: [2, 4, 6, 8]
```

for DIRECT USE →

Code o/p in NaN

<pre>1 const numbers = [1, 2, 3, 4]; 2 const doubled = (numbers * 2); 3 console.log(doubled); // Output: [2, 4, 6, 8]</pre>	<pre>NaN === Code Execution Suc</pre>
---	---------------------------------------

7.5 Filter

Theory: The `filter()` method creates a new array with all elements that pass the test implemented by the provided function.

Example:

```
const numbers = [1, 2, 3, 4, 5];
const evenNumbers = numbers.filter(num => num % 2 === 0);
console.log(evenNumbers);           // Output: [2, 4]
```

7.6 Reduce

Theory: The `reduce()` method executes a reducer function on each element of the array, resulting in a single output value. It takes a callback function and an initial value.

Example:

Javascript

```
const numbers = [1, 2, 3, 4];
const sum = numbers.reduce((accumulator, currentValue) =>
  accumulator + currentValue, 0);
console.log(sum); // Output: 10
```

*Logic

- **reduce ka kaam:**

Yeh array ke elements ko ek single value mein reduce karta hai, yahan hum elements ka sum nikaal rahe hain.

- **Parameters:**

`reduce` method ko ek callback function aur ek initial value pass hoti hai.

Callback Function Parameters:

- **accumulator:** Yeh previous iteration ka result ya initial value hota hai.
- **currentValue:** Yeh array ka current element hota hai.

- **Initial Value:**

Callback function ke pehle argument (`accumulator`) ki initial value set karne ke liye ek value pass ki jati hai. Yahan wo `0` hai.

Callback Function Workflow:

`reduce` har element par callback function ko run karta hai. Iska flow dekhte hain:

Iteration	accumulator (Initial Result)	currentValue (Current Array Element)	Result (accumulator + currentValue)
1	0	1	0 + 1 = 1
2	1	2	1 + 2 = 3
3	3	3	3 + 3 = 6
4	6	4	6 + 4 = 10

7.7 Find

Theory: The `find()` method returns the value of the first element in the array that satisfies the provided testing function. If no values satisfy the testing function, `undefined` is returned.

Example:

```
const numbers = [1, 2, 3, 4, 5];
const found = numbers.find(num => num > 3);
console.log(found); // Output: 4
```

How `find` Works in Iterations:

`find` ek ek karke array ke elements ko check karega:

Iteration	Current Element (num)	Condition (num > 3)	Result
1	1	1 > 3 (False)	Skip
2	2	2 > 3 (False)	Skip
3	3	3 > 3 (False)	Skip
4	4	4 > 3 (True)	Return 4

O/p is 4.

7.8 FindIndex

Theory: The `findIndex()` method returns the index of the first element in the array that satisfies the provided testing function. If no elements satisfy the testing function, `-1` is returned.

Example:

javascript

```
const numbers = [1, 2, 3, 4, 5];
const index = numbers.findIndex(num => num > 3);
console.log(index); // Output: 3
```

logic→

How `findIndex` Works in Iterations:

Iteration	Current Element (num)	Condition (num > 3)	Action
1	1	1 > 3 (False)	Continue
2	2	2 > 3 (False)	Continue
3	3	3 > 3 (False)	Continue
4	4	4 > 3 (True)	Return index 3

- Jaise hi 4 (index 3) condition ko satisfy karta hai (4 > 3), `findIndex` wahan ruk jaata hai aur us element ka index (3) return karta hai.

Chapter 8: Date

8.1 Date and Time

Chapter 8: Date and Time

Understanding how to work with dates and times is essential for many programming tasks, from displaying the current date to calculating durations or time differences. JavaScript provides a built-in `Date` object that simplifies these tasks.

```
console.log("This is date and time tutorial");
let now = new Date();
console.log(now);

let dt = new Date(1000);
console.log(dt);

// let newDate = new Date("2029-09-30");
// console.log(newDate)

// let newDate = new Date(year, month, date, hours, minutes, seconds, milliseconds);
let newDate = new Date(3020, 4, 6, 9, 3, 0);
console.log(newDate)
```

DATE →

```

date_time.html > html > script
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width,
   initial-scale=1.0">
6   <title>Document</title>
7 </head>
8   <script>
9     // let specificDate = new Date('2024-12-21');
10    // console.log(specificDate);
11    let dateComponents = new Date(2024, 12, 21);
12    console.log(dateComponents);
13    let specificDate = new Date('2024-12-21');
14    console.log(specificDate);
15
16  </script>
17 <body>
18   <h1>3456789</h1>
19 </body>
20 </html>

```

Month Starting index in 0;

o/p → ?

```

Tue Jan 21 2025 00:00:00 GMT+0530 (India Standard Time)  date_time.html:12
Sat Dec 21 2024 05:30:00 GMT+0530 (India Standard Time)  date_time.html:14

```

```

let dateComponents = new Date(2024, 11, 21); // 11 का मतलब दिसंबर
console.log(dateComponents);

```

Date Methods in JavaScript

Here is a comprehensive list of the various methods available in the `Date` object in JavaScript, along with their usage and why you might use them:

1. `getFullYear()`

- **Usage:** Retrieves the year (4 digits for 4-digit years) from a date.
- **Why:** To get the current year or the year from a specific date.

```

let now = new Date();
console.log(now.getFullYear()); // Outputs: 2023

```

2. `getMonth()`

- **Usage:** Retrieves the month from a date (0-11, where 0 is January and 11 is December).
- **Why:** To get the current month or the month from a specific date.

javascript

```

let now = new Date();

```

```
console.log(now.getMonth()); // Outputs: 3 (April)
```

explain :-

```
let now6 = new Date();  
console.log(now6.getMonth()); // Outputs: 11 (November)
```

Because current date is 21-12-2024

So o/p is November .

Because month starting is 0 indexing .

3. `getDate()`

- **Usage:** Retrieves the day of the month from a date (1-31).
- **Why:** To get the current day or the day from a specific date.

javascript

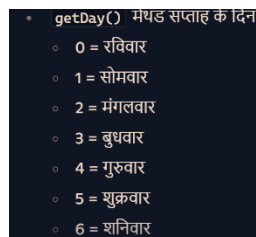
```
let now = new Date();  
console.log(now.getDate()); // Outputs: 21  
because Current date is 21 .
```

4. `getDay()`

- **Usage:** Retrieves the day of the week from a date (0-6, where 0 is Sunday and 6 is Saturday).
- **Why:** To get the current weekday or the weekday from a specific date.

javascript

```
let now = new Date();  
console.log(now.getDay()); // Outputs: 6 (because today is Saturday)
```



5. `setMinutes(minutes, [seconds], [milliseconds])`

- **Usage:** Sets the minutes for a date object, optionally setting seconds and milliseconds.
- **Why:** To change the minutes of a specific date.

javascript

```
let now = new Date();  
now.setMinutes(45, 0, 0);  
console.log(now); // Date object now represents 45 minutes
```

Chapter 9: LocalStorage

9.1 Local Storage Introduction

9.1 Local Storage Introduction in JavaScript

Theory: Local Storage is a web storage feature that allows you to store data on the client's browser with no expiration date. This data persists even after the browser is closed and reopened. Local Storage provides a **simple key-value pair storage system**, making it useful for saving user preferences, session data, and other information that needs to be maintained across page loads.

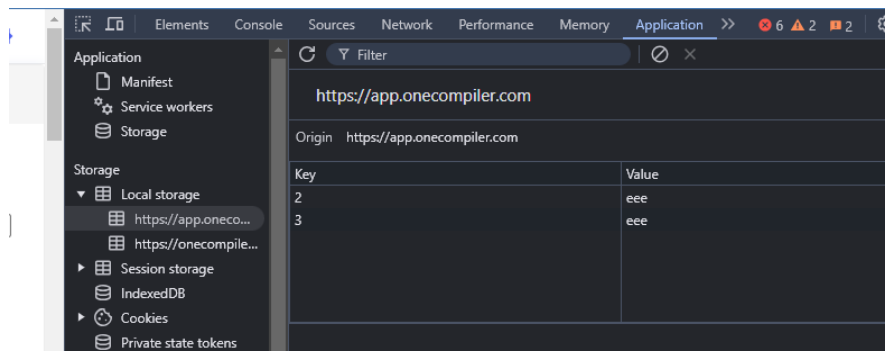
Simple Example :->

```
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Simple Local Storage Example</title>
7 </head>
8 <body>
9   <h1>Simple Local Storage Example</h1>
10  <input type="text" id="key" placeholder="Enter key">
11  <input type="text" id="value" placeholder="Enter value">
12  <button onclick="setItem()">Set Item</button>
13  <button onclick="getItem()">Get Item</button>
14  <button onclick="removeItem()">Remove Item</button>
15  <button onclick="clearStorage()">Clear Storage</button>
16  <p id="output"></p>
17
18  <script>
19    function setItem() {
20      const key = document.getElementById('key').value;
21      const value = document.getElementById('value').value;
22      localStorage.setItem(key, value);
23      document.getElementById('output').textContent = `Set ${key} = ${value}`;
24    }
25
26    function getItem() {
27      const key = document.getElementById('key').value;
28      const value = localStorage.getItem(key);
29      document.getElementById('output').textContent = value ? `Got ${key} = ${value}` : `No value found for key "${key}"`;
30    }
31
32    function removeItem() {
33      const key = document.getElementById('key').value;
34      localStorage.removeItem(key);
35      document.getElementById('output').textContent = `Removed item with key "${key}"`;
36    }
37
38    function clearStorage() {
39      localStorage.clear();
40      document.getElementById('output').textContent = `Cleared all items from Local Storage`;
41    }
42  </script>
43 </body>
44 </html>
```

Simple Local Storage Example

2	eee	Set Item	Get Item
Remove Item	Clear Storage		

Set 2 = eee



Get

Set Item Get Item

Remove Item Clear Storage

Got 3 = eee

Clear Storage :

Simple Local Storage Example

Set Item Get Item

Remove Item Clear Storage

Cleared all items from Local Storage

Another :

<> code_1.html X

<> code_1.html > html > body > script > getData

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>LocalStorage Example</title>
7      <style>
8          body {
9              font-family: Arial, sans-serif;
10             margin: 20px;
11         }
12         input {
13             margin: 5px 0;
14         }
15     </style>
16 </head>
17 <body>
18     <h1>LocalStorage Example</h1>
19
20     <label for="nameInput">Name:</label>
21     <input type="text" id="nameInput"><br>
22
23     <label for="ageInput">Age:</label>
24     <input type="number" id="ageInput"><br>
25
26     <label for="cityInput">City:</label>
27     <input type="text" id="cityInput"><br>
28
29     <button onclick="saveData()">Save Data</button>
30     <button onclick="getData()">Get Data</button>
31     <button onclick="deleteData()">Delete Data</button>
32     <button onclick="clearData()">Clear All Data</button>
33
34 </script>
35 // Function to save data to LocalStorage
36 function saveData() {
37     const name = document.getElementById('nameInput').value;
38     const age = document.getElementById('ageInput').value;
39     const city = document.getElementById('cityInput').value;
40
41     // Create a user object
42     const user = {
43         name: name,
44         age: age,
45         city: city
46     };
47     // Convert the user object to a JSON string and store it in LocalStorage
48     localStorage.setItem('user', JSON.stringify(user));
49     alert('Data saved!');
50 }
51 // Function to get data from LocalStorage
52 function getData() {
53     // Retrieve the user data from LocalStorage
54     const user = JSON.parse(localStorage.getItem('user'));
55
56     if (user) {
57         alert(`Name: ${user.name}\nAge: ${user.age}\nCity: ${user.city}`);
58     } else {
59         alert('No data found!');
60     }
61 }
62
63 // Function to delete specific data from LocalStorage
64 function deleteData() {
65     localStorage.removeItem('user');
66     alert('Data deleted!');
```

```

51 // Function to get data from LocalStorage
52 function getData() {
53     // Retrieve the user data from LocalStorage
54     const user = JSON.parse(localStorage.getItem('user'));
55
56     if (user) {
57         alert(`Name: ${user.name}\nAge: ${user.age}\nCity: ${user.city}`);
58     } else {
59         alert('No data found!');
60     }
61 }
62
63 // Function to delete specific data from LocalStorage
64 function deleteData() {
65     localStorage.removeItem('user');
66     alert('Data deleted!');
67 }
68
69 // Function to clear all data from LocalStorage
70 function clearData() {
71     localStorage.clear();
72     alert('All data cleared!');
73 }
74 </script>
75 </body>
76 </html>
77

```

LocalStorage Example

Name:

Age:

City:

LocalStorage Example

Name:

Age:

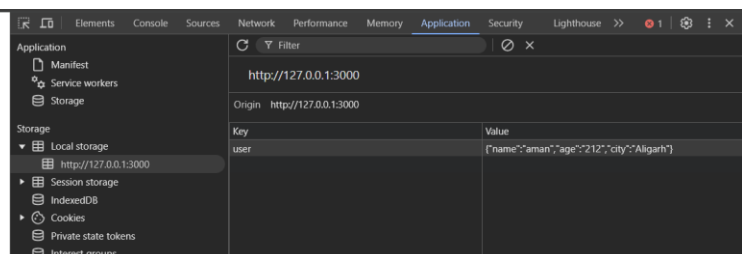
City:

LocalStorage Example

Name:

Age:

City:



Chapter 10: OOP

10.1 Constructor and new Operator

- 10.2 Prototypes
- 10.3 Prototypical Inheritance
- 10.4 ES6 Classes
- 10.5 Setters and Getters
- 10.6 Static Methods
- 10.7 Class Inheritance
- 10.8 Inheritance by Prototypes
- 10.9 Chaining of Methods

Chapter 10: Object-Oriented Programming (OOP)

Simple Example of Objects and Classes

Theory:

1. **Object:** An object is an instance of a class. It encapsulates data (attributes) and methods (functions) that operate on the data.
2. **Class:** A class is a blueprint for creating objects. It defines properties and behaviors that the objects created from the class will have.

```
// Define a class
class Person {
  // Constructor function to initialize object properties
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  // Method to greet
  greet() {
    console.log(`Hello, my name is ${this.name} and I am ${this.age} years old.`);
  }
}

// Create an instance (object) of the Person class
const john = new Person('John', 30);
const jane = new Person('Jane', 25);

// Access object properties and methods
john.greet(); // Output: Hello, my name is John and I am 30 years old.
jane.greet(); // Output: Hello, my name is Jane and I am 25 years old.
```

Without object use for this code , using static keyword →

```
// Define a class
class Person {
  // Constructor function to initialize object properties
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  // Static method to greet
  static greet(name, age) {
    console.log(`Hello, my name is ${name} and I am ${age} years old.`)
  }
}

// Call the static method directly on the class
Person.greet('John', 30); // Output: Hello, my name is John and I am 30 years old
Person.greet('Jane', 25); // Output: Hello, my name is Jane and I am 25 years old
```

- **Static Method:** The `greet` method is defined as a static method using the `static` keyword.
- **Calling the Method:** Static methods are called on the class itself rather than on instances of the class. Therefore, you can call `Person.greet('John', 30)` directly without creating an instance.

Pillars

Inheritance

Inheritance allows a class to inherit properties and methods from another class, promoting code reusability.

```

class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a sound.`);
  }
}

class Dog extends Animal {
  speak() {
    console.log(`${this.name} barks.`);
  }
}

const dog = new Dog("Rex");
dog.speak(); // Output: Rex barks.

```

Here's a breakdown of what each part does:

- **class Person { ... }:** This defines a new class named `Person`.
- **constructor(name) { ... }:** The `constructor` is a special function that gets called whenever a new instance of the `Person` class is created. It takes a parameter `name`.
- **this.name = name;:** Inside the constructor, `this` refers to the new instance being created. This line sets the `name` property of the instance to the value passed as the `name` parameter.

Polymorphism allows objects of different classes to be treated as objects of a common superclass. It enables a single interface to represent different underlying forms (data types).

Theory: Polymorphism allows objects of different classes to be treated as objects of a common superclass. It enables the use of methods with the same name but potentially different implementations in different classes. In JavaScript, polymorphism is typically implemented using method overriding, where a subclass redefines a method from its parent class.

index.js
+
433wtzb3g

```

1 class Animal {
2   speak() {
3     console.log("Animal makes a sound.");
4   }
5 }
6
7 class Dog extends Animal {
8   speak() {
9     console.log("Dog barks.");
10  }
11 }
12
13 class Cat extends Animal {
14   speak() {
15     console.log("Cat meows.");
16   }
17 }
18
19 const animals = [new Dog(), new Cat()];
20 animals.forEach(animal => animal.speak());
21

```

STDIN
Input for the program (Optional)

Output:
Dog barks.
Cat meows.

Abstraction:

Theory: Abstraction is the process of hiding the complex implementation details and showing only the essential features of an object. It allows the developer to focus on the higher-level functionality without worrying about the internal workings. In JavaScript, abstraction is often achieved through the use of classes and methods that provide a simplified interface.

```
index.js + 433wtzb3g
1 class Car {
2   constructor(make, model) {
3     this.make = make;
4     this.model = model;
5   }
6
7   drive() {
8     console.log(`${this.make} ${this.model} is driving`);
9   }
10 }
11
12 const myCar = new Car("Tesla", "Model S");
13 myCar.drive(); // Tesla Model S is driving
14
```

STDIN

Input for the program (Optional)

Output:

Tesla Model S is driving

What is Inheritance?

Inheritance in object-oriented programming (OOP) allows a class (child class) to **inherit** properties and methods from another class (parent class). This helps avoid repeating code and promotes code reuse.

```
index.js + 433wtzb3g
1 // Base class
2 class Animal {
3   constructor(name) {
4     this.name = name;
5   }
6
7   speak() {
8     console.log(`${this.name} makes a noise.`);
9   }
10 }
11
12 // Derived class inheriting from Animal
13 class Dog extends Animal {
14   constructor(name) {
15     super(name); // Call the super class constructor and pass
16   }
17
18   bark() {
19     console.log(`${this.name} barks.`);
20   }
21 }
22
23 // Create an instance of Dog
24 const snoopy = new Dog('Snoopy');
25 snoopy.speak(); // Output: Snoopy makes a noise.
26 snoopy.bark(); // Output: Snoopy barks.
27
```

STDIN

Input for the program (Optional)

Output:

Snoopy makes a noise.
Snoopy barks.

ENCAPSULATION :

10.1 Constructor and new Operator

Constructor and new Operator in JavaScript

In JavaScript, constructors and the `new` operator are used to create and initialize objects. Let's break down these concepts clearly:

1. Constructor in JavaScript

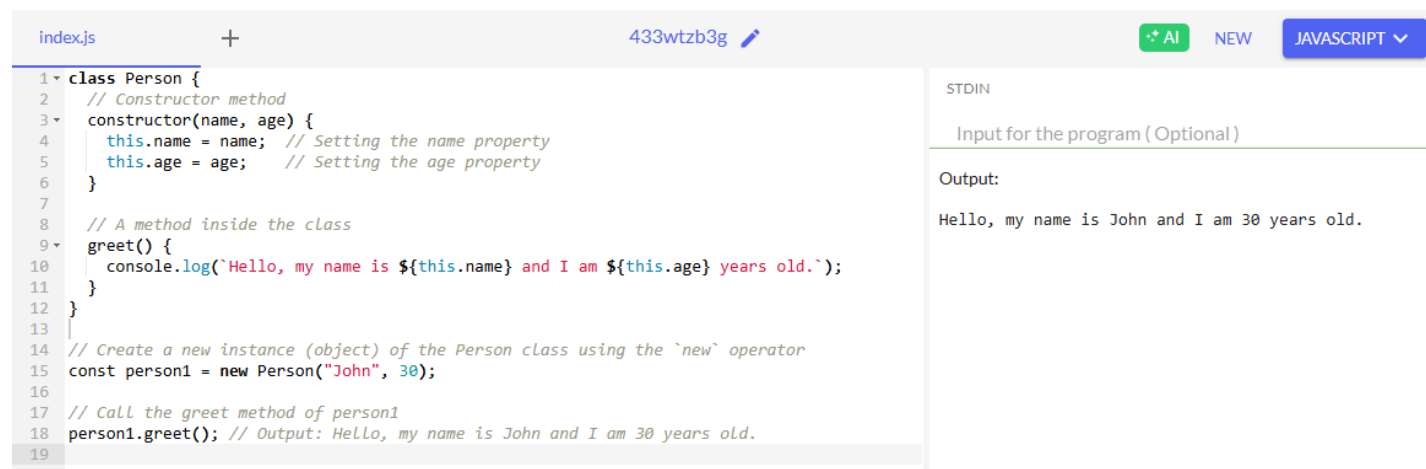
A **constructor** is a special function used to initialize objects created from a class or function. It is automatically called when a new object is created using the `new` keyword.

- **For Classes:** A constructor is a special method in a class that gets called when an object of that class is created.
- **For Functions (Pre-ES6):** Functions can also act as constructors when used with the `new` keyword.

2. The `new` Operator

The `new` operator is used to create an instance of a class or a function (used as a constructor). When `new` is used:

1. A new empty object is created.
2. The constructor function is called with `this` referring to the newly created object.
3. The newly created object is returned from the constructor (unless the constructor explicitly returns another object).



```
index.js + 433wtzb3g AI NEW JAVASCRIPT v
1 class Person {
2   // Constructor method
3   constructor(name, age) {
4     this.name = name; // Setting the name property
5     this.age = age;   // Setting the age property
6   }
7
8   // A method inside the class
9   greet() {
10    console.log(`Hello, my name is ${this.name} and I am ${this.age} years old.`);
11  }
12 }
13
14 // Create a new instance (object) of the Person class using the `new` operator
15 const person1 = new Person("John", 30);
16
17 // Call the greet method of person1
18 person1.greet(); // Output: Hello, my name is John and I am 30 years old.
19
```

STDIN

Input for the program (Optional)

Output:

Hello, my name is John and I am 30 years old.

10.2 Prototypes

Concept: Prototypes are a fundamental feature in JavaScript that allows objects to inherit properties and methods from other objects. Every JavaScript object has a prototype, which is another object that the original object inherits properties from.

Why Use Prototypes? :

- **Inheritance:** Allows objects to share properties and methods, promoting code reuse.
- **Efficient Memory Usage:** Methods can be defined once on the prototype and shared by all instances, rather than defining them on each instance separately.

- **Dynamic Updates:** Adding or modifying methods on the prototype automatically updates all instances that inherit from it.

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}

// Adding a method to the prototype
Person.prototype.greet = function() {
  console.log(`Hello, my name is ${this.name}`);
};

const person1 = new Person('John', 30);
person1.greet(); // Outputs: Hello, my name is John
```

Without Prototype

When we don't use prototypes, each object instance has its own copy of methods, which can lead to redundant memory usage.

```
function Car(model, year) {
  this.model = model;
  this.year = year;
  this.drive = function() {
    console.log(`${this.model} is driving`);
  };
}

const car1 = new Car('Toyota', 2020);
const car2 = new Car('Honda', 2019);
const car3 = new Car('Ford', 2018);

car1.drive(); // Outputs: Toyota is driving
car2.drive(); // Outputs: Honda is driving
car3.drive(); // Outputs: Ford is driving
```

With Prototype →

```
function Car(model, year) {
  this.model = model;
  this.year = year;
}

// Adding the drive method to the prototype
Car.prototype.drive = function() {
  console.log(`${this.model} is driving`);
};

const car1 = new Car('Toyota', 2020);
const car2 = new Car('Honda', 2019);
const car3 = new Car('Ford', 2018);

car1.drive(); // Outputs: Toyota is driving
car2.drive(); // Outputs: Honda is driving
car3.drive(); // Outputs: Ford is driving
```

Another Example : →

```
1- function Car(model, year) {
2   this.model = model;
3   this.year = year;
4 }
5- Car.prototype.drive = function() {
6   console.log(`${this.model} is driving`);
7 };
8
9 // Stop method
10- Car.prototype.stop = function() {
11   console.log(`${this.model} has stopped`);
12 };
13 // Honk method
14- Car.prototype.honk = function() {
15   console.log(`${this.model} is honking: Beep! Beep!`);
16 };
17 const car1 = new Car('Toyota', 2020);
18 const car2 = new Car('Honda', 2019);
19 const car3 = new Car('Ford', 2018);
```

o/p →

```
20
21 car1.drive(); // Outputs: Toyota is driving
22 car1.stop(); // Outputs: Toyota has stopped
23 car1.honk(); // Outputs: Toyota is honking: Beep! Beep!
24
25 car2.drive(); // Outputs: Honda is driving
26 car2.stop(); // Outputs: Honda has stopped
27 car2.honk(); // Outputs: Honda is honking: Beep! Beep!
28
29 car3.drive(); // Outputs: Ford is driving
30 car3.stop(); // Outputs: Ford has stopped
31 car3.honk(); // Outputs: Ford is honking: Beep! Beep!
32
```

```
Toyota is driving
Toyota has stopped
Toyota is honking: Beep! Beep!
Honda is driving
Honda has stopped
Honda is honking: Beep! Beep!
Ford is driving
Ford has stopped
Ford is honking: Beep! Beep!
```

10.3 Prototypical Inheritance

Theory: Prototypical Inheritance is a core concept in JavaScript where objects can inherit properties and methods from other objects. This is achieved through the prototype chain, which allows objects to reference other objects' properties and methods.

Prototypical Inheritance enables:

1. **Inheritance:** Objects inherit properties and methods from their prototypes.
2. **Prototype Chain:** If a property or method is not found on an object, JavaScript looks up the prototype chain to find it.
3. **Dynamic Nature:** Prototypes can be modified at runtime, and changes are reflected in all inheriting objects
4. **Prototypical Inheritance (Theory and Code)**
5. Prototypical inheritance is a fundamental concept in JavaScript. It allows objects to inherit properties and methods from other objects. This differs from classical inheritance (used in languages like Java or C++) because, in JavaScript, every object has a prototype (another object), and it can inherit properties and methods from that prototype.

Without use →

Example without Using `Object.create()` (Prototype Property):

In this example, we won't use `Object.create()`, but we'll manually set up the prototype inheritance.

```
1
2- function Animal(species) {
3   this.species = species;
4 }
5- Animal.prototype.speak = function() {
6   console.log(this.species + ' makes a sound');
7 };
8 // Child object constructor
9- function Dog(name) {
10  Animal.call(this, 'Dog'); // Call the parent constructor
11  this.name = name;
12 }
13 // Set up the inheritance manually
14 Dog.prototype = Object.create(Animal.prototype); // Inherit methods
   from Animal
15 Dog.prototype.constructor = Dog; // Correct the constructor reference
16 // Add additional methods specific to Dog
17- Dog.prototype.bark = function() {
18   console.log(this.name + ' barks');
19 };
20 // Create an instance of Dog
21 const dog = new Dog('Buddy');
22 dog.speak(); // Output: "Dog makes a sound"
23 dog.bark(); // Output: "Buddy barks"
24
```

Use →

<pre>1 // Parent object 2- const animal = { 3 species: 'Unknown', 4- speak() { 5 console.log(this.species + ' makes a sound'); 6 } 7 }; 8 9 // Creating a child object 10 const dog = Object.create(animal); // dog inherits from animal 11 dog.species = 'Dog'; // Overriding the species property for dog 12 dog.speak(); // Output: "Dog makes a sound" 13 14 // Creating another child object 15 const cat = Object.create(animal); 16 cat.species = 'Cat'; 17 cat.speak(); // Output: "Cat makes a sound" 18</pre>	<pre>Dog makes a sound Cat makes a sound === Code Execution Successful ===</pre>
---	---

The code is very easy and simple


```

function Animal(eats) {
  this.eats = eats;
  this.walk = function() {
    console.log("Animal walks");
  };
}

function Rabbit(jumps) {
  this.jumps = jumps;
  this.eats = true; // Inherited property added manually
  this.walk = function() {
    console.log("Animal walks"); // Inherited method added manually
  };
}

const rabbit1 = new Rabbit(true);

console.log(rabbit1.eats); // true, manually added
console.log(rabbit1.jumps); // true, own property
rabbit1.walk(); // Animal walks, manually added method

```

10.4 ES6 Classes

Theory: ES6 (ECMAScript 2015) introduced a class syntax that provides a clearer and more concise way to create objects and handle inheritance. Although classes in JavaScript are syntactic sugar over the prototypical inheritance model, they make the code more readable and maintainable.

Features of ES6 Classes:

1. **Class Declaration:** Use the `class` keyword to define a class.
2. **Constructor Method:** A special method for creating and initializing objects created within a class.
3. **Inheritance:** Use the `extends` keyword to create a subclass.
4. **Super:** Use the `super` keyword to call the constructor and methods of the parent class.

main.js	Output
<pre> 1 // Base class 2 class Animal { 3 constructor(name) { 4 this.name = name; 5 } 6 walk() { 7 console.log(`\${this.name} walks`); 8 } 9 } 10 // Subclass 11 class Rabbit extends Animal { 12 constructor(name, color) { 13 super(name); // Call the constructor of the parent class 14 this.color = color; 15 } 16 jump() { 17 console.log(`\${this.name} jumps`); 18 } 19 } 20 const rabbit1 = new Rabbit("Bunny", "white"); 21 rabbit1.walk(); // Bunny walks 22 rabbit1.jump(); // Bunny jumps 23 // Check instance 24 console.log(rabbit1 instanceof Rabbit); // true 25 console.log(rabbit1 instanceof Animal); // true </pre>	<pre> Bunny walks Bunny jumps true true === Code Exec </pre>

Using Super Keyword (simple Code and simple logic)

main.js	Output
<pre> 1 // Base class 2 class Animal { 3 constructor(name) { 4 this.name = name; 5 } 6 makeSound() { 7 console.log(`\${this.name} makes a sound`); 8 } 9 } 10 // Subclass 11 class Dog extends Animal { 12 constructor(name, breed) { 13 super(name); // Call the constructor of the parent class 14 this.breed = breed; 15 } 16 bark() { 17 console.log(`\${this.name} barks: Woof! Woof!`); 18 } 19 } 20 const dog1 = new Dog('Buddy', 'Golden Retriever'); 21 dog1.makeSound(); // Outputs: Buddy makes a sound 22 dog1.bark(); // Outputs: Buddy barks: Woof! Woof! 23 </pre>	<pre> Buddy makes a sound Buddy barks: Woof! Woof! === Code Execution Success </pre>

10.5 Setters and Getters

Theory: Setters and getters are special methods in JavaScript that allow you to define how properties of an object are accessed and updated. They are useful for adding extra logic when reading or setting property values.

```

1 class User {
2   constructor(name) {
3     this._name = name;
4   }
5   get name() {
6     return this._name;
7   }
8
9   set name(value) {
10    if (value.length > 0) {
11      this._name = value;
12    } else {
13      console.log('Name cannot be empty');
14    }
15  }
16 }
17 const user = new User('Alice');
18 console.log(user.name); // Outputs: Alice
19 user.name = 'Bob';
20 console.log(user.name); // Outputs: Bob
21 user.name = ''; // Outputs: Name cannot be empty
22

```

Alice
Bob
Name cannot be empty
=== Code Execution Successful

Difference →

Aspect	Getter	Setter
Purpose	Retrieve the value of a property	Set the value of a property
Syntax	Defined using the <code>get</code> keyword	Defined using the <code>set</code> keyword
Usage	Accessed like a regular property (e.g., <code>object.property</code>)	Assigned like a regular property (e.g., <code>object.property = value</code>)
Logic	Can include logic for processing or transforming the value	Can include logic for validation or transformation before setting the value
Read-Only	Can make properties read-only by not defining a corresponding setter	Cannot make properties read-only
Controlled Access	Allows controlled access to	Allows controlled modification of

Only get use →

```

class User {
  constructor(name) {
    this._name = name;
  }

  // Getter for the name property
  get name() {
    return this._name;
  }
}

const user = new User('Alice');
console.log(user.name); // Outputs: Alice

```

Only value Set :→

main.js	Output
<pre> 1- class User { 2- constructor(name) { 3- this._name = name; 4- } 5- // Setter for the name property 6- set name(value) { 7- if (value.length > 0) { 8- this._name = value; 9- } else { 10- console.log('Name cannot be empty'); 11- } 12- } 13- } 14 const user = new User('Alice'); 15 // Setting the name property using the setter 16 user.name = 'Bob'; 17 console.log(user._name); // Outputs: Bob 18 19 // Trying to set an empty name 20 user.name = ''; // Outputs: Name cannot be empty 21 console.log(user._name); // Outputs: Bob (remains unchanged) 22 </pre>	<pre> Bob Name cannot be empty Bob === Code Execution Succ </pre>

10.6 Static Methods

10.7 Class Inheritance

10.6 Static Methods

Theory: Static methods are functions that are defined on the class itself rather than on instances of the class. They are called directly on the class and do not require an instance to be invoked. Static methods are often used for utility functions, such as creating or configuring objects.

main.js	Output
<pre> 1- class Calculator { 2- static add(a, b) { 3- return a + b; 4- } 5- static subtract(a, b) { 6- return a - b; 7- } 8- static multiply(a, b) { 9- return a * b; 10- } 11- static modulus(a, b) { 12- return a % b; 13- } 14- } 15 console.log(Calculator.add(10, 5)); // Outputs: 15 16 console.log(Calculator.subtract(10, 5)); // Outputs: 5 17 console.log(Calculator.multiply(10, 5)); // Outputs: 50 18 // console.log(Calculator.divide(10, 5)); // Outputs: 2 19 console.log(Calculator.modulus(10, 5)); // Outputs: 0 20 </pre>	<pre> 15 5 50 0 === Code Ex </pre>

10.7 Class Inheritance

Theory: Class inheritance allows one class to inherit properties and methods from another class. This is achieved using the `extends` keyword. The subclass can also have its own additional properties and methods or override the parent class's methods.

```
1- class Animal {
2-   constructor(name) {
3-     this.name = name;
4-   }
5-
6-   makeSound() {
7-     console.log(`${this.name} makes a sound`);
8-   }
9- }
10- class Dog extends Animal {
11-   constructor(name, breed) {
12-     super(name); // Call the parent class constructor
13-     this.breed = breed;
14-   }
15-   bark() {
16-     console.log(`${this.name} barks: Woof! Woof!`);
17-   }
18- }
19- const dog = new Dog('Buddy', 'Golden Retriever');
20- dog.makeSound(); // Outputs: Buddy makes a sound
21- dog.bark();      // Outputs: Buddy barks: Woof! Woof!
22-
```

Buddy makes a sound
Buddy barks: Woof! Woof!

=== Code Execution Successful ===

10.9 Chaining of Methods

Theory: Method chaining is a technique that allows you to call multiple methods on the same object consecutively in a single statement. Each method returns the object itself, allowing further methods to be called on it.

```
main.js
1- class Calculator {
2-   constructor(value = 0) {
3-     this.value = value;
4-   }
5-   add(number) {
6-     this.value += number;
7-     return this; // Return the object itself
8-   }
9-   subtract(number) {
10-    this.value -= number;
11-    return this; // Return the object itself
12-  }
13-   multiply(number) {
14-    this.value *= number;
15-    return this;
16-  }
17-   divide(number) {
18-    if (number !== 0) {
19-      this.value /= number;
20-    }
21-    return this; // Return this for chaining
22-  }
23-   result() {
24-    console.log(this.value);
25-    return this.value;
26-  }
27- }
28- const calc = new Calculator();
29- calc.add(10).subtract(2).multiply(3).divide(2).result(); // Outputs:
30- 12
```

12

=== Code Execution Successful ===

```
const calc = new Calculator();
calc.add(10).subtract(2).multiply(3).divide(2).result(); // Outputs: 12
```

- `const calc = new Calculator();` : यह `Calculator` का एक नई इंस्टेंस बनाता है जिसका प्रारंभिक `value` 0 होता है।
- `calc.add(10).subtract(2).multiply(3).divide(2).result();` : यह विधियाँ चेनिंग तरीके से कॉल की जाती हैं।
 - `add(10)` : प्रारंभिक `value` 0 में 10 जोड़ता है → वर्तमान `value` = 10।
 - `subtract(2)` : 10 से 2 घटाता है → वर्तमान `value` = 8।
 - `multiply(3)` : 8 को 3 से गुणा करता है → वर्तमान `value` = 24।
 - `divide(2)` : 24 को 2 से भाग करता है → वर्तमान `value` = 12।
 - `result()` : वर्तमान `value` (12) को कंसोल में प्रदर्शित करता है और इसे वापस लौटाता है।

PROJECT →

TODO List:

```
index.html    styles.css    script.js    +    
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>To-Do List</title>
7      <link rel="stylesheet" href="style.css">
8  </head>
9  <body>
10     <div class="todo-container">
11         <h1>My To-Do List</h1>
12         <input type="text" id="new-task" placeholder="Add a new task...">
13         <button id="add-task">Add Task</button>
14         <ul id="task-list"></ul>
15     </div>
16     <script src="script.js"></script>
17 </body>
18 </html>
19 |
```

```
1 body {
2   font-family: Arial, sans-serif;
3   display: flex;
4   justify-content: center;
5   align-items: center;
6   height: 100vh;
7   background-color: #f4f4f4;
8   margin: 0;
9 }
10
11 .todo-container {
12   background: white;
13   padding: 20px;
14   border-radius: 5px;
15   box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
16   text-align: center;
17   width: 300px;
18 }
19
20 h1 {
21   margin-bottom: 20px;
22 }
23
24 input[type="text"] {
25   width: calc(100% - 40px);
26   padding: 10px;
27   margin-bottom: 10px;
28   border: 1px solid #ddd;
29   border-radius: 5px;
30 }
31
32 button {
33   padding: 10px;
34   border: none;
35   background-color: #007bff;
36   color: white;
37   border-radius: 5px;
38   cursor: pointer;
39 }
40
41 button:hover {
42   background-color: #0056b3;
43 }
44
45 ul {
46   list-style: none;
47   padding: 0;
48 }
```

49

50 li {

51 padding: 10px;

52 border-bottom: 1px solid #ddd;

53 display: flex;

54 justify-content: space-between;

55 }

56

```
index.html    styles.css    script.js    +    [icon]
1 document.getElementById('add-task').addEventListener('click', function() {
2     const taskInput = document.getElementById('new-task');
3     const taskValue = taskInput.value;
4
5     if (taskValue) {
6         const li = document.createElement('li');
7         li.textContent = taskValue;
8         const deleteButton = document.createElement('button');
9         deleteButton.textContent = 'Delete';
10        deleteButton.addEventListener('click', function() {
11            li.remove();
12        });
13
14        li.appendChild(deleteButton);
15        document.getElementById('task-list').appendChild(li);
16        taskInput.value = '';
17    }
18 });
19
```

o/p ->

My To-Do List

- hello
- 3456



✔ What is a Constructor?

A **constructor** is a special function used to **create and initialize an object** in object-oriented programming.

In **JavaScript**, the constructor is:

- Defined using the `constructor()` method inside a class.
-

🔗 Think of it like this:

A constructor is like the **blueprint's builder** — it fills in the object's details when you create it.

◆ Constructor Definition (in simple Hindi + English):

Constructor ek special function hota hai jo tab automatically chalta hai **jab object banate ho** kisi class ka.

Iska kaam hota hai **object ke andar values set karna** (initialization).

With Constructor ➔

main.js	Output
<pre>1- class Car { 2- constructor(brand, speed) { 3- this.brand = brand; 4- this.speed = speed; 5- } 6- 7- drive() { 8- console.log(`\${this.brand} is driving at \${this.speed} km/h.`); 9- } 10- } 11- 12- const car1 = new Car("Toyota", 120); 13- car1.drive(); // Toyota is driving at 120 km/h. 14-</pre>	Toyota is driving at 120 km/h. === Code Execution Successful ===

w/o ➔

main.js	Output
<pre>1- const car2 = { 2- brand: "Honda", 3- speed: 100, 4- drive: function () { 5- console.log(`\${this.brand} is driving at \${this.speed} km/h.`); 6- } 7- }; 8- 9- car2.drive(); // Honda is driving at 100 km/h. 10-</pre>	Honda is driving at 100 km/h. === Code Execution Successful ===



✓ What is Inheritance?

Inheritance means one class (child) can use properties and methods of another class (parent).

Think of it like: "**Bachcha apne maa-baap ke features le sakta hai.**"

? Types of Inheritance in JavaScript

JavaScript supports inheritance mainly through **prototypes** and **classes (ES6)**.

Here are the main types:

1. Single Inheritance

One child class inherits from one parent.

main.js	Output
<pre>1 class Animal { 2 speak() { 3 console.log("Animal speaks"); 4 } 5 } 6 class Dog extends Animal { 7 bark() { 8 console.log("Dog barks"); 9 } 10 } 11 const d = new Dog(); 12 d.speak(); // Animal speaks 13 d.bark(); // Dog barks 14</pre>	<pre>Animal speaks Dog barks === Code Executed ===</pre>

2. Multilevel Inheritance

Child → Parent → Grandparent

main.js		Output
<pre>1 class LivingBeing { 2 live() { 3 console.log("I am alive"); 4 } 5 } 6 7 class Animal extends LivingBeing { 8 speak() { 9 console.log("Animal speaks"); 10 } 11 } 12 13 class Dog extends Animal { 14 bark() { 15 console.log("Dog barks"); 16 } 17 } 18 19 const d = new Dog(); 20 d.live(); // I am alive 21 d.speak(); // Animal speaks 22 d.bark(); // Dog barks 23</pre>		<pre>I am alive Animal speaks Dog barks === Code Executing</pre>

3. Hierarchical Inheritance

One parent → multiple children

main.js		Output
<pre> 1 class Vehicle { 2 start() { 3 console.log("Engine starts"); 4 } 5 } 6 class Car extends Vehicle { 7 drive() { 8 console.log("Car is driving"); 9 } 10 } 11 class Bike extends Vehicle { 12 ride() { 13 console.log("Bike is riding"); 14 } 15 } 16 17 const c = new Car(); 18 c.start(); // Engine starts 19 c.drive(); // Car is driving 20 21 const b = new Bike(); 22 b.start(); // Engine starts 23 b.ride(); // Bike is riding 24 </pre>	   Share  Run	<pre> Engine starts Car is driving Engine starts Bike is riding </pre> === Code Execution

⚠ 4. Multiple Inheritance (Not directly supported)

JavaScript **does not support** true multiple inheritance (i.e., one class inheriting from multiple parents) but you can simulate it using **mixins**.

✓ Mixin Example (Workaround for Multiple Inheritance):

```
main.js  [ ] [ ] [ ] Share Run Output
1- let Flyable = {
2-   fly() {}
3-   console.log("I can fly");
4- }
5- };
6- let Swimmable = {
7-   swim() {
8-     console.log("I can swim");
9-   }
10- };
11- class Bird {
12-   chirp() {
13-     console.log("Chirp chirp");
14-   }
15- }
16-
17- Object.assign(Bird.prototype, Flyable, Swimmable);
18-
19- const b = new Bird();
20- b.chirp(); // Chirp chirp
21- b.fly();   // I can fly
22- b.swim();  // I can swim
23-
```

Inheritance Type	Supported?	Example Class Usage
Single	✓ Yes	class B extends A
Multilevel	✓ Yes	A → B → C
Hierarchical	✓ Yes	One parent → many children
Multiple	✗ (via mixins)	Object.assign() workaround

✓ What is Encapsulation?

Encapsulation means **hiding internal details** of an object and only exposing **what's necessary**.

Think of it like a **TV remote** — you don't need to know what's inside, just how to use it.



Encapsulation is the process of **binding data and methods** together and **restricting direct access** to some of the object's internal details.

main.js

Share

Run

Output

```

1- class TVRemote {
2  #volume = 10; // 🔒 private variable
3  // ✅ Getter (to access safely)
4-  getVolume() {
5    return this.#volume;
6  }
7  // ✅ Setter (to modify with control)
8-  setVolume(v) {
9    if (v >= 0 && v <= 100) {
10     this.#volume = v;
11-   } else {
12     console.log("❌ Volume must be between 0 and 100");
13   }
14 }
15 }
16 const remote = new TVRemote();
17
18 console.log(remote.getVolume()); // ✅ 10
19 remote.setVolume(50); // ✅ set to 50
20 console.log(remote.getVolume()); // ✅ 50
21
22 remote.setVolume(200); // ❌ Invalid, will not change
23 console.log(remote.getVolume()); // ✅ still 50
24

```

10
50
❌ Volume must be between 0 and 100
50

=== Code Execution Successful ===

Symbol	Type	Compares	Converts Type?	Language
==	Loose	Value	✓ Yes	JavaScript
===	Strict	Value + Type	✗ No	JavaScript
.equals()	Method	Value	✗ No	Java only

```

5 === "5" // false ❌ (number ≠ string)
true === 1 // false ❌
5 === 5 // true ✅

```

→ constructor

```
main.js  [Icons] [Share] [Run] Output
1 class Car {
2   constructor(name) {
3     this.name = name; // ✅ initialization
4   }
5
6   showName() {
7     console.log(`🚗 Car name is ${this.name}`);
8   }
9 }
10
11 const c = new Car("BMW"); // 🏠 constructor call
12 c.showName(); // 🚗 Car name is BMW
13
```

🚗 Car name is BMW
=== Code Execution Successful



Abstraction

```
main.js  [Icons] [Share] [Run] Output
1 class Car {
2   #engineStarted = false; // 🔒 private detail
3
4   start() {
5     this.#engineStarted = true;
6     console.log("🚗 Car started");
7   }
8
9   stop() {
10    this.#engineStarted = false;
11    console.log("🛑 Car stopped");
12  }
13 }
14
15 const c = new Car();
16 c.start(); // 🚗 Car started
17 c.stop(); // 🛑 Car stopped
18
```

🚗 Car started
🛑 Car stopped
=== Code Execution Su

Feature	Constructor	Abstraction
Purpose	Object banate hi data set karna	Use karna asaan banana (andar ka logic hide)
Kab use hota hai?	<code>new ClassName(...)</code> ke time	Object banne ke baad
Focus	Initialization (setup)	Hiding complexity
Visible to User	Yes (<code>constructor(...)</code>)	No (andar ka logic nahi dikhata)

✓ What is Polymorphism?

Poly = many

Morph = form

So, **Polymorphism** means:

"Ek hi cheez ka alag-alag form ya behavior."

◆ Real-life Example:

- “draw()” function:
 - Circle ke liye **draw a circle**
 - Square ke liye **draw a square**
 - Rectangle ke liye **draw a rectangle**

Type	Description
1. Method Overriding	Same method name, different classes
2. Duck Typing	JS ka unique style — function call based on behavior

main.js

Share

Run

```
1 class Animal {
2   speak() {
3     console.log("🔊 Animal makes a sound");
4   }
5 }
6 class Dog extends Animal {
7   speak() {
8     console.log("🐶 Dog barks");
9   }
10 }
11 class Cat extends Animal {
12   speak() {
13     console.log("🐱 Cat meows");
14   }
15 }
16 // 📌 Create objects
17 const a = new Animal();
18 const d = new Dog();
19 const c = new Cat();
20
21 // 📌 Call the same method
22 a.speak(); // 🔊 Animal makes a sound
23 d.speak(); // 🐶 Dog barks
24 c.speak(); // 🐱 Cat meows
25
```

Output

🔊 Animal makes a sound

🐶 Dog barks

🐱 Cat meows

=== Code Execution Success

◆ Duck Typing Simple Definition:

"Agar koi object kisi method ko support karta hai, to hum use wo method call kar lete hain — bina ye jaane ki wo kis class ka hai."

Iska naam hai:

"Agar wo duck ki tarah chalta hai, aur duck ki tarah awaaz karta hai, to wo duck hi hoga."

```
main.js  [Full Screen] [Dark Mode] [Share] [Run]

1 function startMachine(machine) {
2   machine.start();
3 }
4 const car = {
5   start: () => console.log("🚗 Car engine started"),
6 };
7
8 const computer = {
9   start: () => console.log("💻 Computer booted up"),
10 };
11
12 startMachine(car);      // 🚗 Car engine started
13 startMachine(computer); // 💻 Computer booted up
14
```

Output

```
🚗 Car engine started
💻 Computer booted up

=== Code Execution Successful ===
```

Concept	Status	Tera Progress
Constructor	✓	<code>constructor()</code> se object init kiya
Encapsulation	✓	private variable + getter/setter (like TV Remote)
Abstraction	✓	sirf zaroori method dikhaye, logic chhupaya
Inheritance	✓	<code>extends</code> keyword, base → derived
Polymorphism	✓	same method name, different behavior
Duck Typing	✓	JS special — method hona chahiye, class check nahi